IoT Internet of Things

Hartmut Seitter

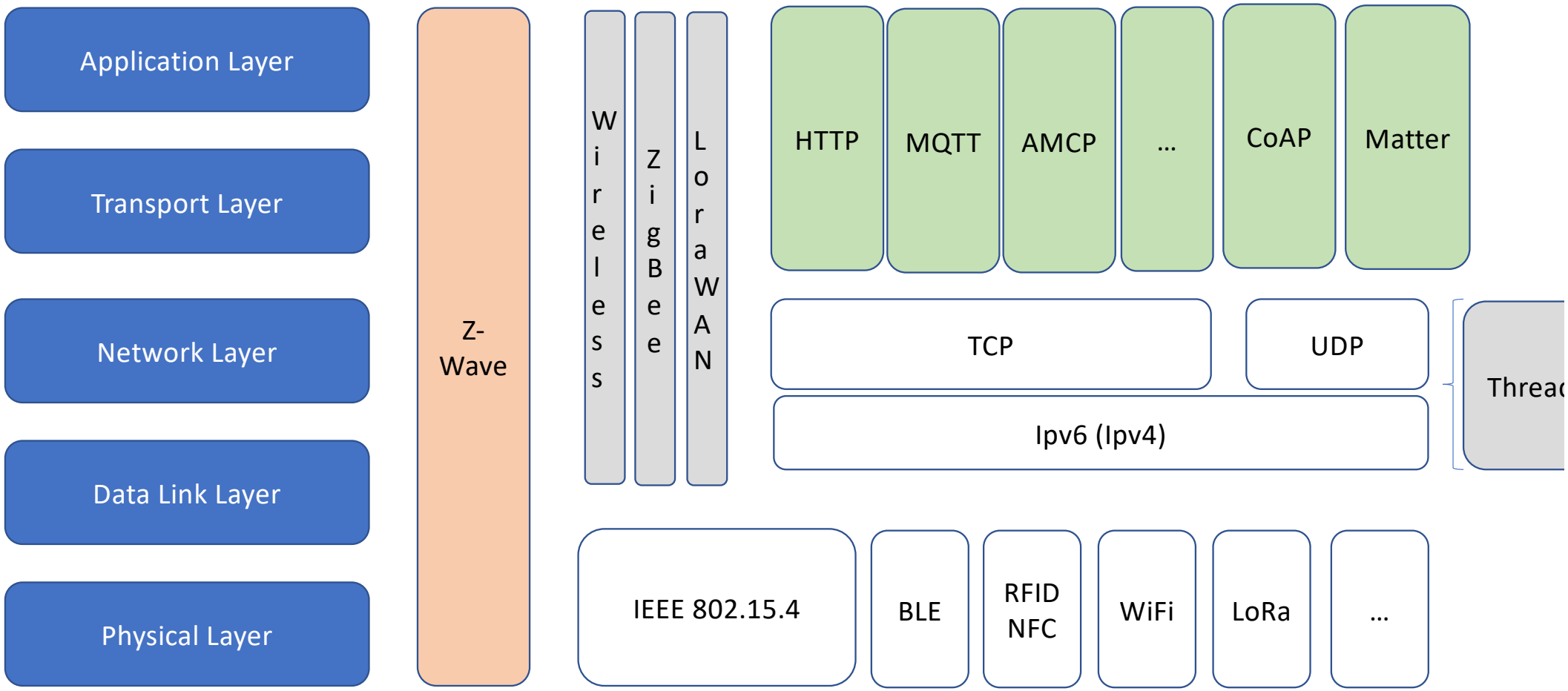
Agenda – Topics which should be covered in this course

- Start with IoT Lab
- Connectivity and Networking
- IoT Networks and Communication frameworks
  - MQTT
  - LoRaWAN
  - CoAp
  - ZigBee
  - Thread, Matter
- Embedded power supplies, energy harvesting and constrains
- Internet principals, routing for IoT
- Information security and privacy concepts
- Big Data and Machine Learning

IoT Protocols

- **Remember in IoT we have**
  - **Sensors which has an interface to the microcontroller**
    - **Analog signal, digital signal, I2C bus, onewire bus, BLE and other …**
  - **Microcontroller which communicate with a gateway or direct to a server using**
    - **MQTT (over TCP), Wifi, LTE, ZigBee, LoRaWAN, other**
    - **Most times they communicate wireless, however communication via Ethernet is also used depending on use cases**
  - **Gateway which receive the data from the microcontroller and forward it to a Server Application – most time TCP/IP based**
  - **Server application which receive the data for pre-/post-processing**

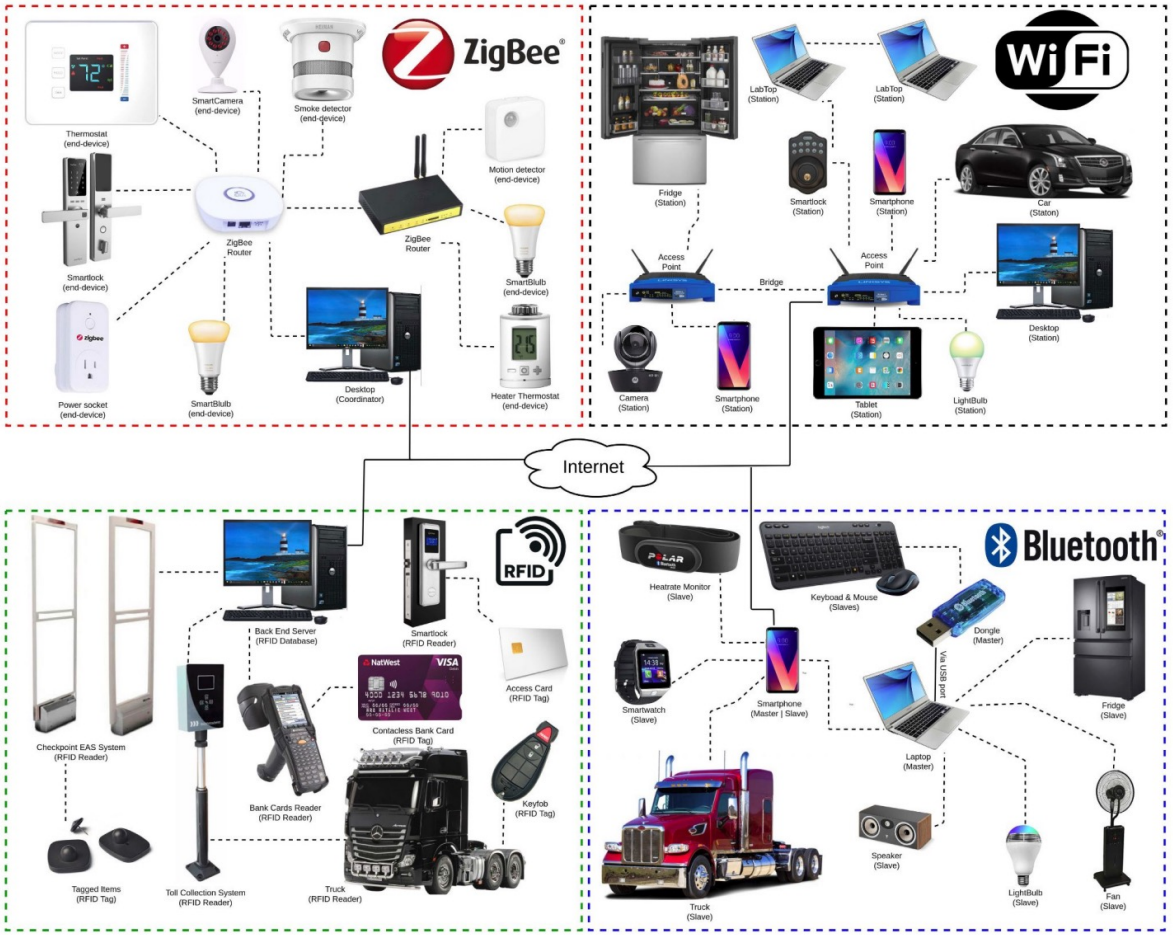
The IoT Network Stack



Source: https://www.mdpi.com/2079-9292/9/1/111/htm

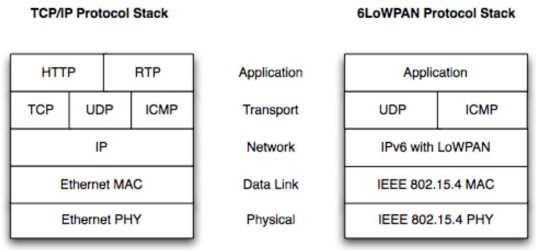
With my own modification

The Physical / Data Link Layer



Source: K. Lounis, M. Zulkernine: Attacks and Defenses in Short-Range Wireless Technologies for IoT

Physical layer IEEE 802.15.4



- **Standard for low power IoT networks**

- Physical & data link layers
- Base for ZigBee, Thread, WirelessHART
- 6LoWPAN relies on it (*IPv6 over Low Power Wireless Personal Network*)
- Support for multiple topologies (star, mesh)
- 64-bit MAC addresses, 16-bit short addresses

- Small packet size – 128 bytes including MAC, 103 bytes payload
- Data rates between 20kbps and 250kbps
- Range: – Usually between 10m and 30m – Some strong transceivers
- Hundreds of meters / km
- Line of sight
- ZigBee Pro

	802.15.4	WiFi (802.11)	Bluetooth
Data rate	250 kbps	up to GBit/s	1–3 Mbps
Range	10–100 m	up to 100 m	~10 m
Power consumption	very low	high	medium
Typical use	IoT/Sensors	Internet access	Audio/HID

Data link layer

- **Responsible for data transfer between two directly connected nodes**

- Takes the raw bit from physical layer and organize it into frames
- Controls access
- Detects transmission errors - retransmission
- Addressing at local network level

Network layer

- **Responsible for end-to-end delivery of data across multiple hops**

- Logical addresses independent of hardware
- Routing – finding the best path through the network
- Passing packets hop-by-hop towards to destination
- Splitting large packages to fit lower layer limits

- Summary
- *The network layer solves the problem of* ***"how do I reach a node that is not my direct neighbor?"*** *— it provides logical addressing and routing so that a sensor in a mesh network can communicate with a server on the internet, hop by hop.*

Transport layer

- **Provides end-to-end communication between applications on two devices**

- **While the network layer delivers packets between machines, the transport layer delivers data between specific processes (applications) on those machines**.

- Port addressing Identifies the correct application on a device
- Segmentation Splits data into segments, reassembles at destination
- Flow control Prevents sender from overwhelming the receiver
- Error control Detects lost/corrupted segments and retransmits
- Connection management Establishes and terminates connections (TCP)

- *The transport layer answers the question* ***"which application on this device should receive this data?"*** *— it multiplexes communication between processes using ports, and optionally adds reliability on top of the raw network delivery.*

Transport layer – some notes on these two protocols

UDP — User Datagram Protocol
- Connectionless, no handshake
- No guaranteed delivery, no ordering
- Very lightweight → ideal for IoT/sensor data
- Example: a temperature sensor sending periodic readings

TCP — Transmission Control Protocol
- Connection-oriented (3-way handshake)
- Guaranteed delivery, ordered, flow-controlled
- Higher overhead → less common in constrained IoT devices
- Example: firmware update, configuration transfer

In the Context of IoT

TCP is often too heavy for constrained devices:
- 3-way handshake consumes energy
- Retransmission timers cause delays
- Header overhead wastes the small 802.15.4 payload

UDP is the default choice in IoT stacks:

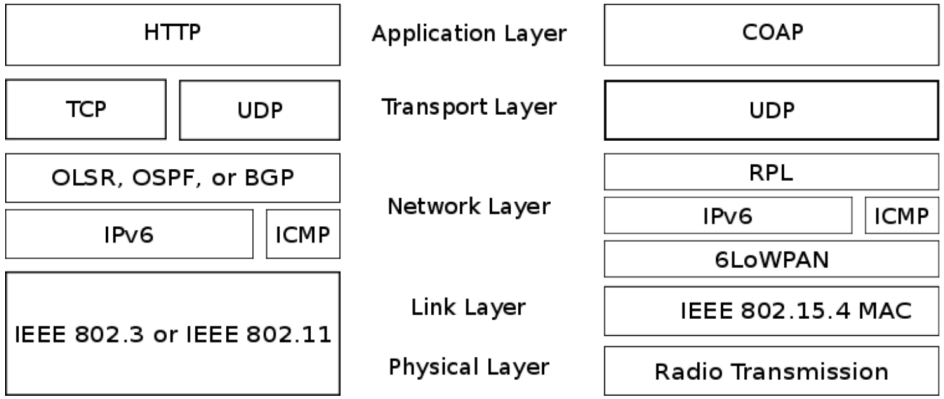
Application Layer Protocol

The application layer is where **actual data exchange between devices and services** happens. In IoT, the challenge is: classic protocols like HTTP are too heavy for constrained devices.

Key question: ***How do tiny, battery-powered sensors communicate efficiently with servers, cloud platforms, or each other?***

- HTTP – HyperText Transfer Protocol
- CoAP – Constrained Application Protocol
- WebSocket
- MQTT – Message Queue Telemetry Transport
- XMPP – eXtensible Messaging and Presence Protocol
- DDS – Data Distribution Service
- AMQP – Advanced Message Queuing Protocol
- Matter

Different protocol stack models or the future



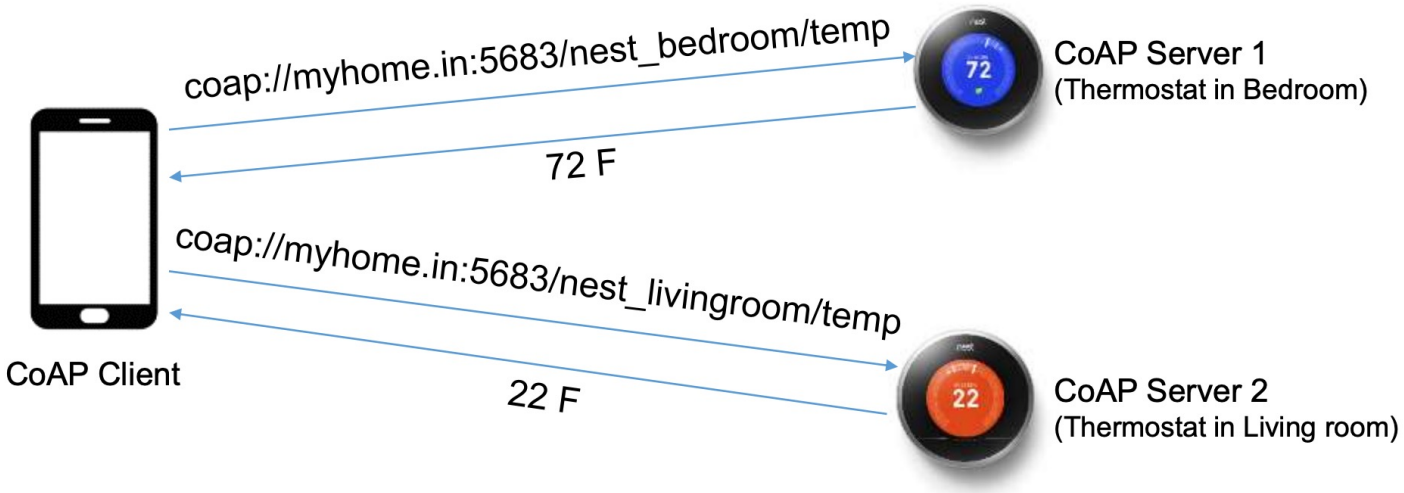
In IoT there is no single best protocol — the right choice depends on topology (broker vs. direct), transport constraints (TCP vs. UDP), and use case (telemetry vs. device management). Understanding the trade-offs is more important than knowing any single protocol.

And another IoT application protocol - CoAP

- **CoAP -> Constrained Application Protocol**
  - **It is a specialized web transfer protocol for use with constrained nodes and constrained networks in IoT**
  - **The protocol is designed for machine-to-machine communications and IoT applications such as smart energy and building automation**

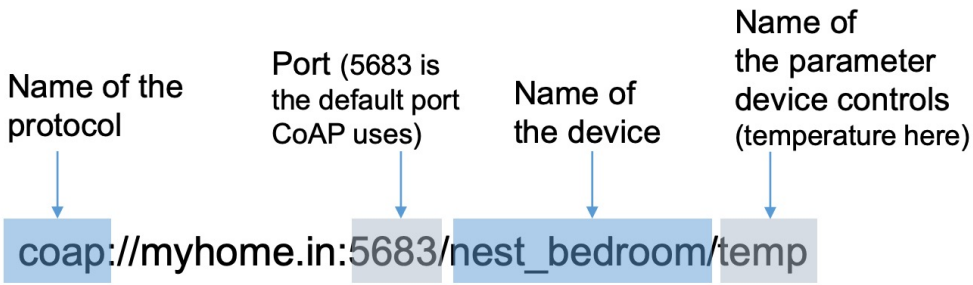
  - Inspired from HTTP – similar to HTTP. CoAP also uses a request response model.
  - Can be transparently mapped to HTTP - However, CoAP also provides features that go beyond HTTP such as native push notifications and group communication.
  - From a developer point of view, CoAP feels very much like HTTP. Obtaining a value from a sensor is not much different from obtaining a value from a Web API.
  - REST model for small devices - It implements the REST architectural style
  - Made for billions of nodes with very less memory - The Internet of Things will need billions of nodes, many of which will need to be inexpensive. CoAP has been designed to work on microcontrollers with as low as 10 KiB of RAM and 100 KiB of code space.
  - Designed to use minimal resources, both on the device and on the network. Instead of a complex transport stack, it gets by with UDP on IP. A 4-byte fixed header and a compact encoding of options enables small messages that cause no or little fragmentation on the link layer.
  - Like HTTP, CoAP can carry different types of payloads, and can identify which payload type is being used. CoAP integrates with XML, JSON, CBOR, or any data format of your choice.
  - Discovery integrated - CoAP resource directory provides a way to discover the properties of the nodes (devices/things in IoT) on your network

CoAP in 2 foils – just to be aware it exist and it is used in IoT



CoAP Methods

Get
Post
Put
Delete



MQTT vs CoAP

The following distinctions should be made between the MQTT and CoAP protocols:

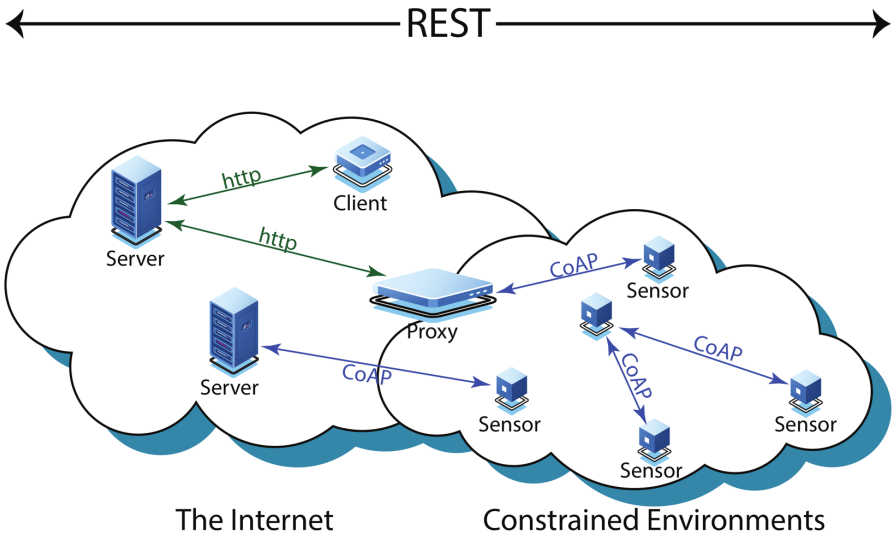
	CoAP	MQTT
Communication Model	Request/Response	Publish/Subscribe
Transport Protocol	User Datagram Protocol (UDP)	Transmission Control Protocol (TCP)
Security	DTLS over UDP	SSL/TLS over TCP
Bandwidth Usage	Very minimal, ideal for very small networks	Efficient, but requires additional data
Reliability	Built-in acknowledgments possible but optional	High reliability through TCP
Architecture	Direct communication between devices	Communication via a central broker

CoAP strengths and weaknesses

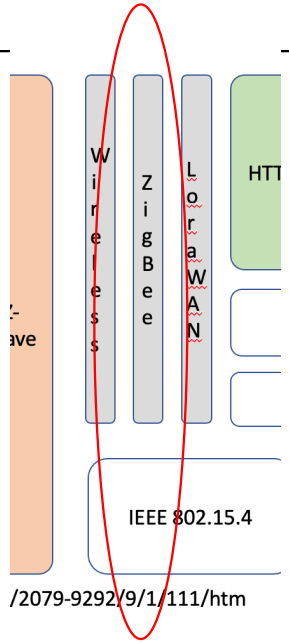
- The Constrained Application Protocol offers numerous advantages for resource-constrained and connected devices but also comes with certain technical challenges. Below is a summary of the main strengths and weaknesses of the CoAP standard.
- **Strengths of CoAP:**
- Lightweight and requires minimal memory and processing power
- Ideal for battery-powered devices
- Low overhead saves bandwidth and energy
- Easy integration into existing IoT systems thanks to REST-based architecture
- Supports multicast for addressing multiple devices simultaneously
- Asynchronous communication capability increases efficiency in connected systems
-
- **Weaknesses of CoAP:**
- Uses UDP without built-in error correction, which can lead to packet loss
- Requires its own mechanisms for reliability, such as acknowledgment messages
- Not directly compatible with traditional web protocols like HTTP
- Data integration requires proxies, increasing system complexity

ESP32 - Arduino Framework and CoAP

- For CoAP on ESP32, you can use the ESP-IDF CoAP client/server examples to connect to a server, send GET/PUT/POST/DELETE requests, and handle secure DTLS connections.



A few words about ZigBee



- **Good to know ☺**
  - **The technique that honey bees uses to communicate new-found food sources to other members of the colony is referred as the ZigBee principle.**

- **The ZigBee standard is currently an 'open' standard only to those that are part of the ZigBee Alliance. For this reason, the ZigBee standard was not used to implement the application layer.**

- **ZigBee** is an IEEE 802.15.4-based specification for a suite of high-level communication protocols used to create personal area networks with small, low-power digital radios, such as for home automation, medical device data collection, and other low-power low-bandwidth needs, designed for small scale projects which need wireless connection. Hence, ZigBee is a low-power, low data rate, and close proximity (i.e., personal area) wireless ad hoc network.

ZigBee general characteristics

- Data rate of 20kbps up to 250kbps
- Intended for 2,45GHz, 868MHz and 915Mhz band
- Data rates
  - 250kbps on 2,45Ghz
  - 40kbps on 915 Mhz
  - 20kbps on 868Mhz

- Star or Peer to Peer network topologies
- Support for low latency devices
- Handshaking
- Low power consumption
- Extremely low duty cycle (<0,1%)

ZigBee what does it do?

- **Designed for wireless controls and sensors**
- **Operates in Personal Area Networks (PAN's) and device to device networks**
- **Connectivity between small packet devices**
- **Control of lights, switches, thermostats, appliances, ….**

- **On devices there are three different nodes available**
  - **ZigBee coordinator node**
  - **ZigBee full function node**
  - **ZigBee reduced function node**

- **The mode of operation is**
  - **Beacon and**
  - **Non-beacon**

ZigBee Coordinator node

- **Root of the network tree**
- **Acts as a bridge to other networks**
- **Stores info about the network**
- **There is only one ZigBee coordinator node for the complete network**
- **Acts as the trust centre and is the repository for security keys.**

- **Set the permissions**
- **Allow user access**
- **Coordinate area network**

- **It starts the network with predefined parameter**
- **works in the same way as ZigBee router after start**

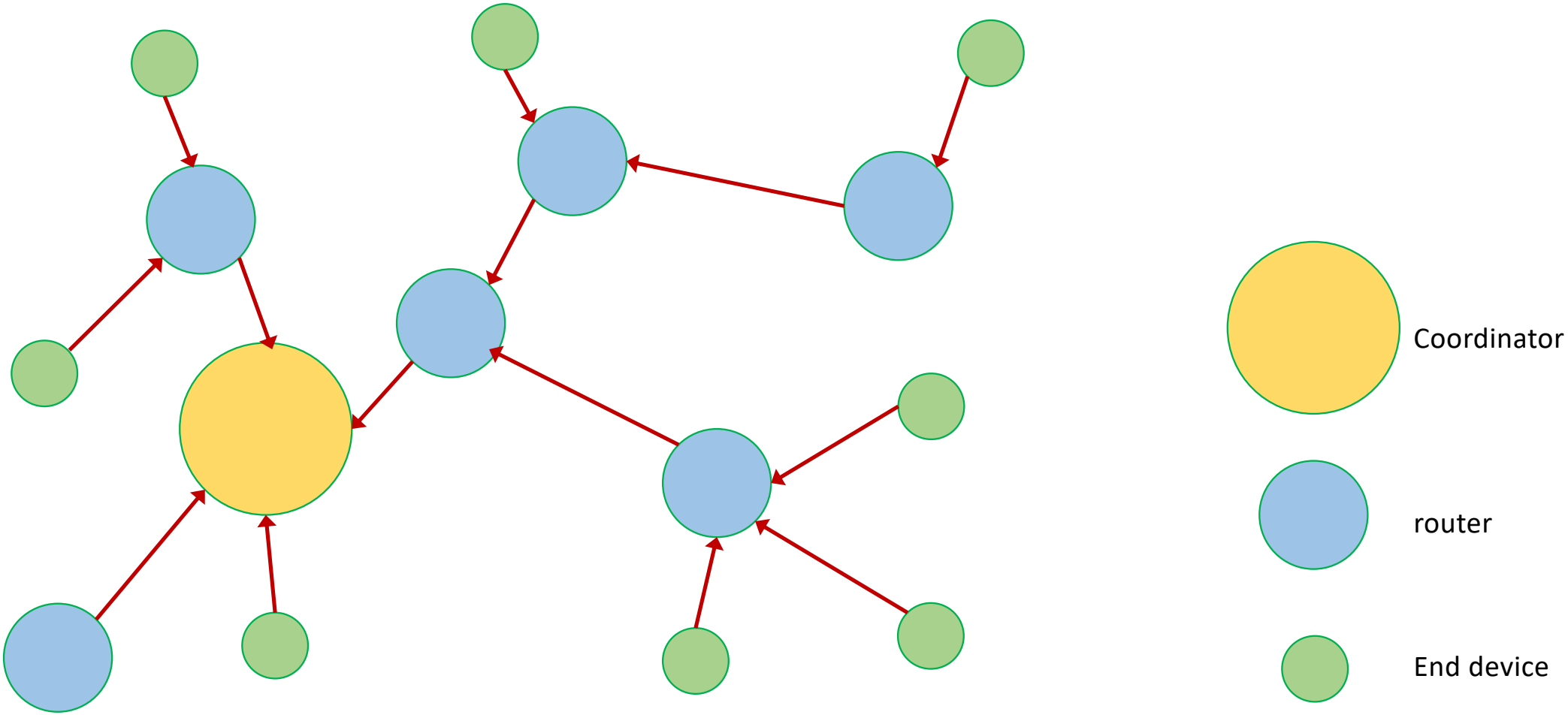
ZigBee full function node

- **An intermediate router in the network**
- **Transmitting and Receiving data from other devices**
- **Needs less memory than ZigBee coordinator node**
- **Cheaper**
- **Can operate on all topologies**

ZigBee reduced function node

- **Also called End Device**
- **Device capable of talking in the network**
- **Can’t relay data from other devices**
- **Cheaper than full function node**

Device types and roles



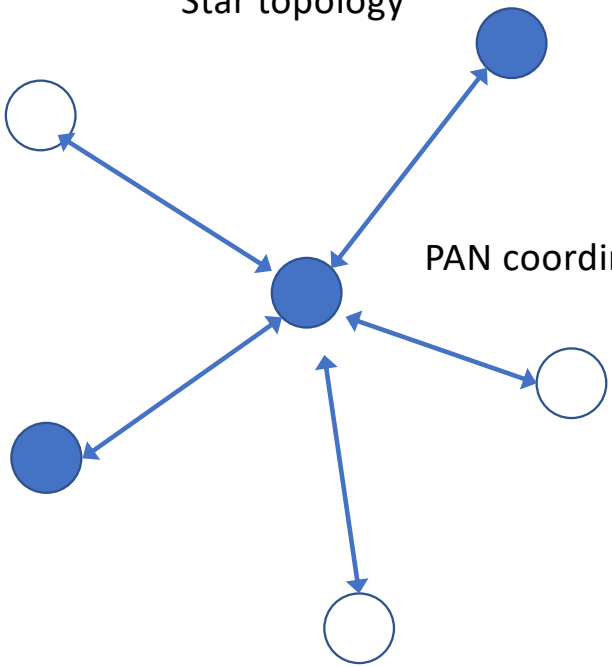
ZigBee mode of operations

- **Beacon mode:**
  - **In beacon enabled networks, the special network node call ZigBee Router transmit periodically beacons to confirm their presence to other network nodes. Nodes may sleep between beacons, thus lowering their duty cycle and extending their battery life**
- **Non beacon mode**
  - **In non-beacon enabled networks, an CSMA/CA (Carrier-sense multiple access with collision avoidance) channel access mechanism is used.**

    ZigBee routers typical have their receivers continuedly active to monitor this CSMA/CA channel

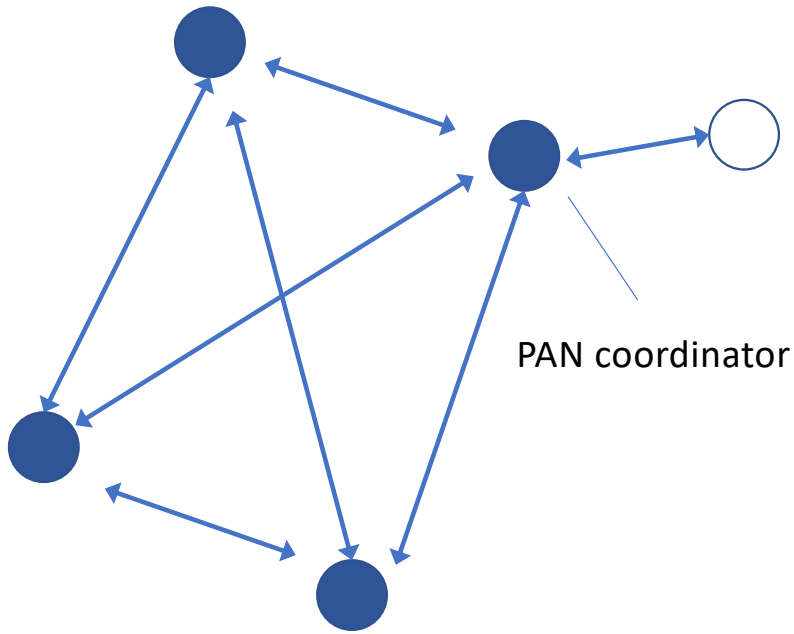
ZigBee network topology

Star topology



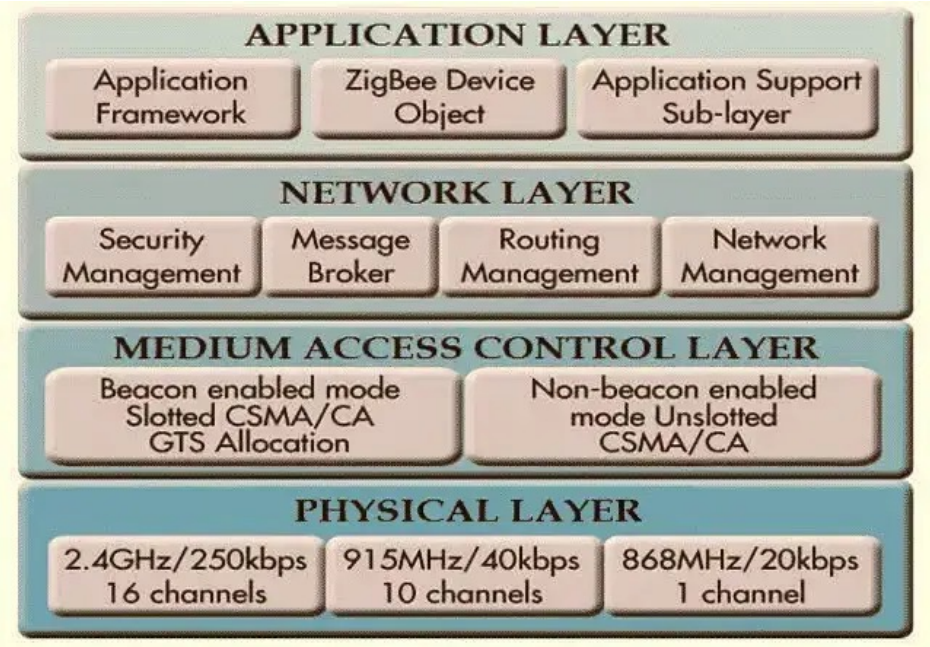
Reduced function device

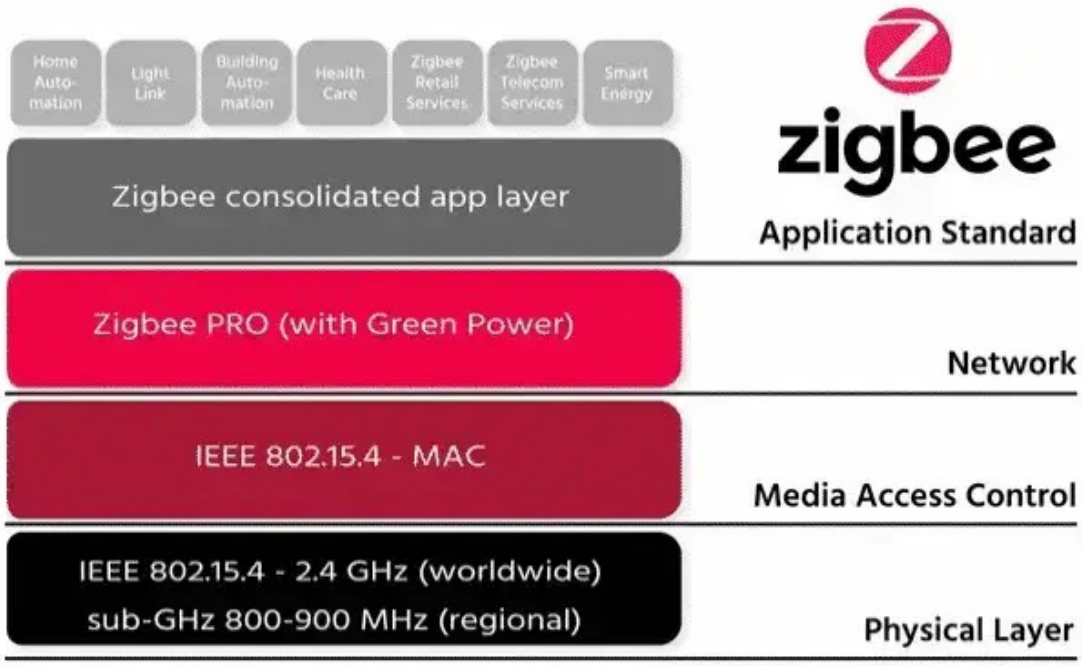
Peer to Peer topology



Full function device

ZigBee architecture





Device addressing

- All devices have a IEEE address
- Short addresses can be allocated
- Addressing modes:

  - Network + device identifier
  - Source / destination identifier
  - Source / destination cluster tree + device identifier

  - Every device has a 64 bit mac address
  - 16 bit network address is used to route packets within the network

ZigBee

- **Approximately 400 companies are in the ZigBee alliance**

- **More than 2500 devices available which support the ZigBee protocol**

  - **E.g. Philips hue**
  - **Sonoff**
  - **Ikea**
  - **….**

ZigBee and Bluetooth comparison

Features	BLE	ZigBee
Power	Very low	low
Complexity	Complex	Simple
Nodes/master	It depends (no limit) however smartphone up to 10	64000
Latency	10 …30 msec	30msec … 1sec
Range	…60m	70m … 300m
Extendibility	No	Yes
Datarate	1 mbps	250kbps
Security	64bit, 128bit	128 bit AES and app. layer

ZigBee device compatibility repository

- **https://ZigBee.blakadder.com/**
- **A lot of devices are available which support ZigBee**
- **There are also adapter available to bridge the ZigBee network to other networks**
- **e.g. the ZigBee2MQTT**

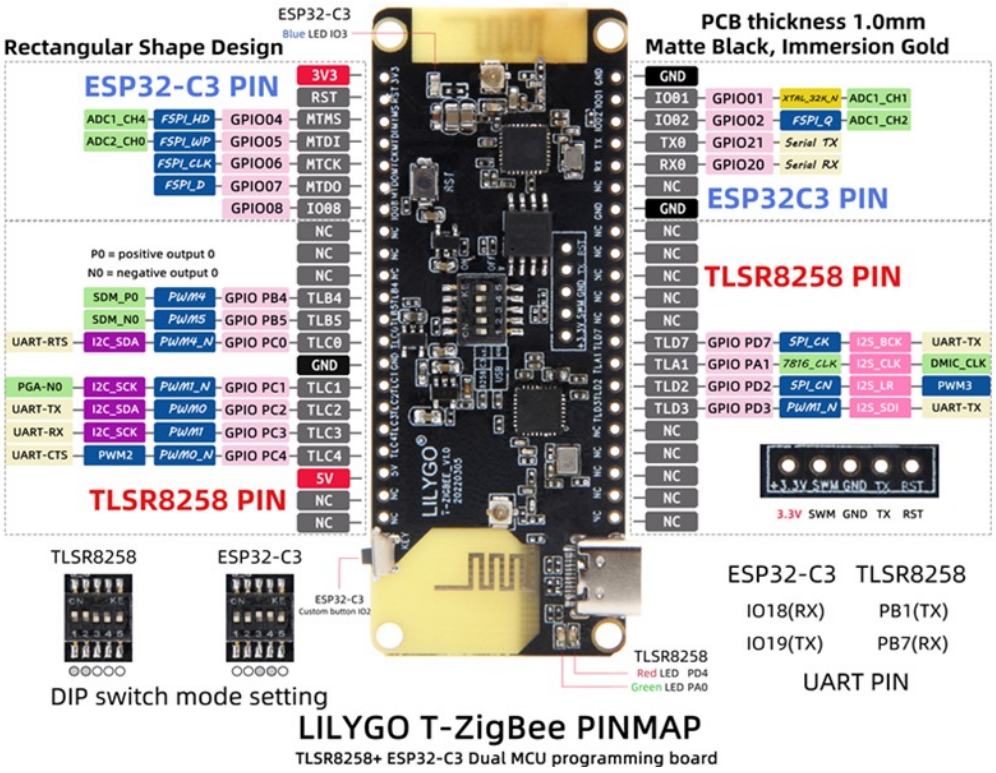
In order to use Zigbee2MQTT we need the following hardware:

1. **A Zigbee Adapter** which is the interface between the Computer (or Server) where you run Zigbee2MQTT and the Zigbee radio communication. Zigbee2MQTT supports a variety of adapters with different kind of connections like USB, GPIO or remote via WIFI or Ethernet. Recommended adapters have a chip starting with CC2652 or CC1352. See supported Adapters.
2. **A Server** where you would run Zigbee2MQTT. Most Raspberry-Pi models are known to work but you can run it on many computers and platforms including Linux, Windows an MacOS. It should have an MQTT broker installed. Mosquitto (Tutorial for Raspberry-Pi) is the recommended MQTT broker but others should also work fine.
3. One or more **Zigbee Devices** which will be paired with Zigbee2MQTT.

ZigBee ESP32 development board

- **e.g.** LILYGO T-Zigbee Dual MCU Development Board ESP32-C3 TLSR8258 Ultra Low Power IOT WiFi BLE Smart Control Module (T-Zigbee T- U2T Kit)

- TLSR8258
  - Low power consumption: □
    - Whole Chip RX mode: 5.3mA
    - Whole Chip TX mode: 4.8mA @ 0dBm with DCDC
    - Deep sleep with external wakeup (without SRAM retention): 0.4uA



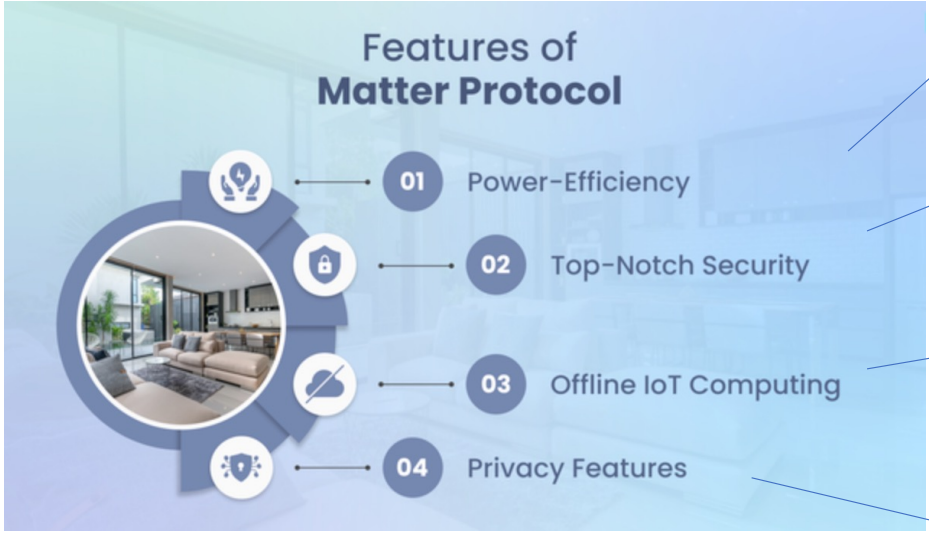
ZigBee — Strengths & Weaknesses

STRENGTH	DETAILS
Mesh network	Self-healing, large coverage through multi-hop routing
Low power	End devices can sleep for years on a single battery
Proven technology	On the market since 2004, deployed in millions of devices
Large ecosystem	Philips Hue, IKEA Tradfri, many industrial devices

WEAKNESS	DETAILS
No IP	Not directly routable to the internet — a gateway is always required
Interoperability	Profile fragmentation before ZigBee 3.0, many proprietary extensions
Being displaced	Thread + Matter taking over the smart home market
Proprietary ecosystem	Dependency on ZigBee Alliance (now CSA)

A view words about the Tread and Matter protocol

- **Launched November 2022**
- **Matter is an open-source standard**
- **More than 170 companies will make their devices ‘matter friendly’**



Matter uses the thread protocol and thread devices are low power devices that uses a minimum power to transmit data.

The Matter protocol takes a layered approach to ensure security.

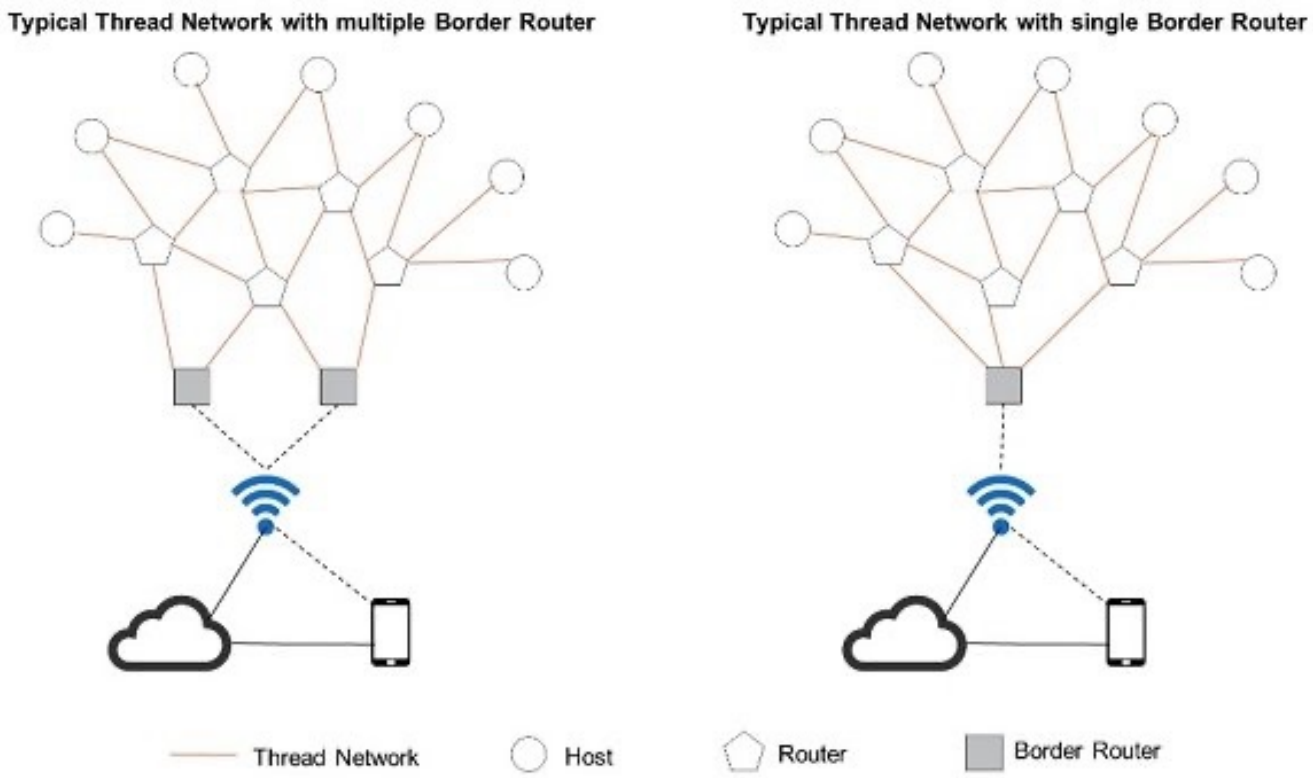
With the ability to communicate with each other directly, devices do not need access to the internet. This feature of Matter makes it more secure, especially for devices like security cameras and smart locks.

All the devices are required to provide proof of identiy. It can be an attestation keypair coupled with an X.509 certification provided by a trusted certificate authority.

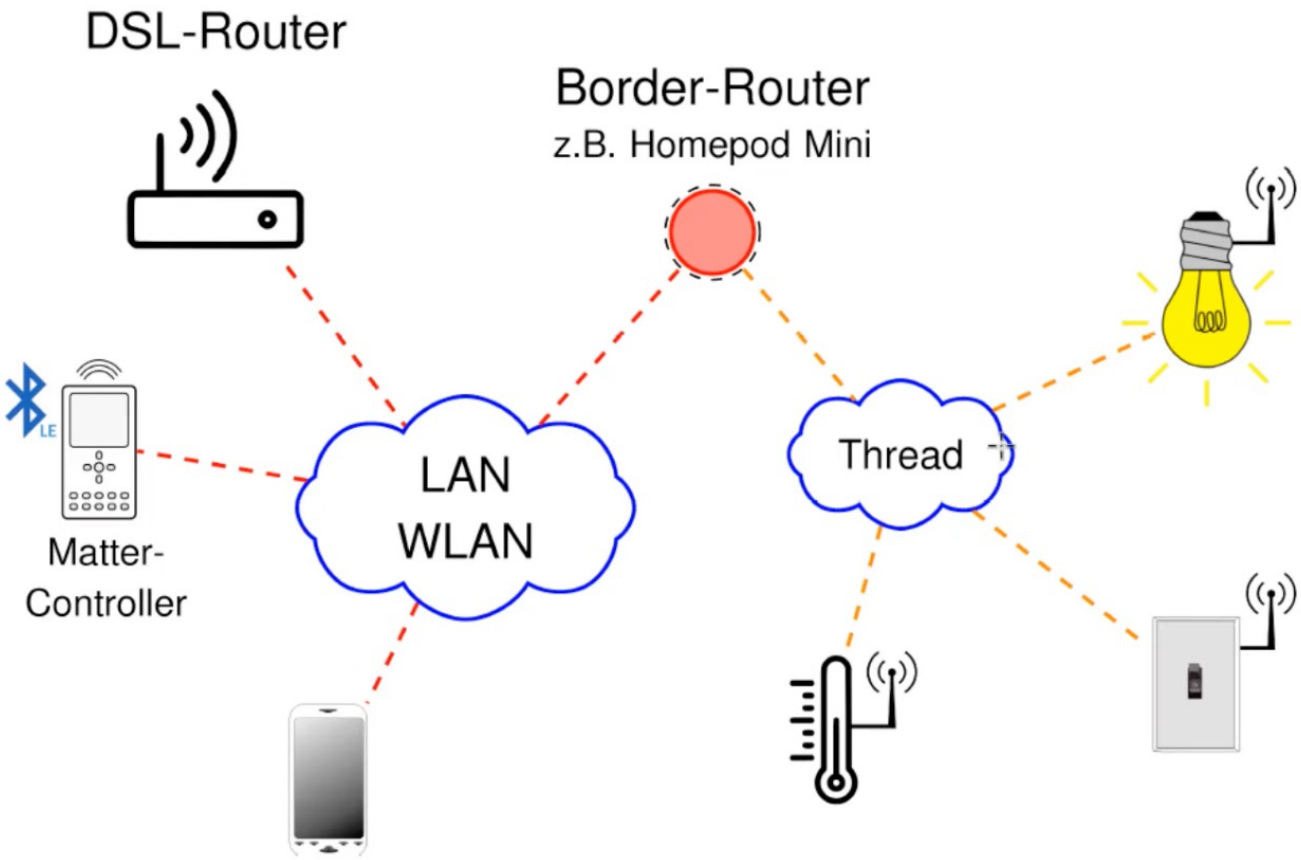
Matter and Thread

- **Matter Protocol:**
  - **Interoperability**: Matter is an application layer protocol that focuses on device communication and interoperability across different ecosystems (like Apple HomeKit, Google Assistant, Amazon Alexa).
  - **Standardization**: It standardizes how devices communicate, ensuring they can work together seamlessly regardless of the manufacturer.
  - **Device Types and Services**: Matter defines specific types of devices (like lights, locks, sensors) and services, allowing developers to implement them consistently across platforms.
- **Thread Protocol:**
  - **Networking Technology**: Thread is a low-power, wireless mesh networking protocol specifically designed for connecting low-power devices in IoT applications.
  - **IPv6 Support**: Thread is built on IPv6, allowing devices to communicate directly over the Internet without the need for a central hub.
  - **Reliability and Scalability**: Its mesh networking capability means devices can communicate with each other directly, enhancing reliability and extending the range by allowing devices to pass messages to one another.
  - **Low Power Consumption**: Thread is designed for battery-operated devices, enabling long battery life while maintaining secure and reliable communication.
- **Relationship between Matter and Thread:**
  - **Matter over Thread**: Matter can be implemented over the Thread networking protocol, allowing Matter-enabled devices to benefit from the low-power and reliable communication characteristics provided by Thread.
  - **Interoperability in a Mesh Network**: When powered by Thread, Matter devices can easily form a mesh network, allowing them to communicate efficiently with each other, even if they are not directly connected to a central hub.
  - **Enhanced Connectivity**: The combination of Matter for device interoperability and Thread for efficient networking allows for a comprehensive and future-proof smart home ecosystem.

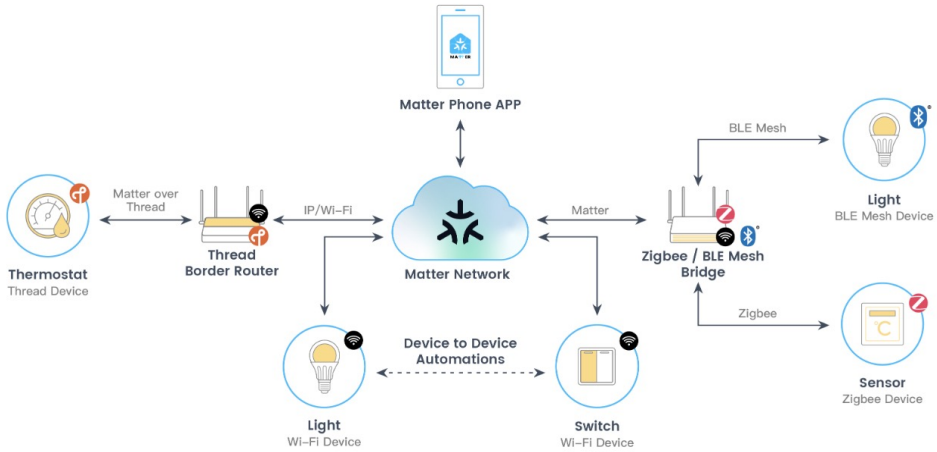
Typical Thread Network Topology

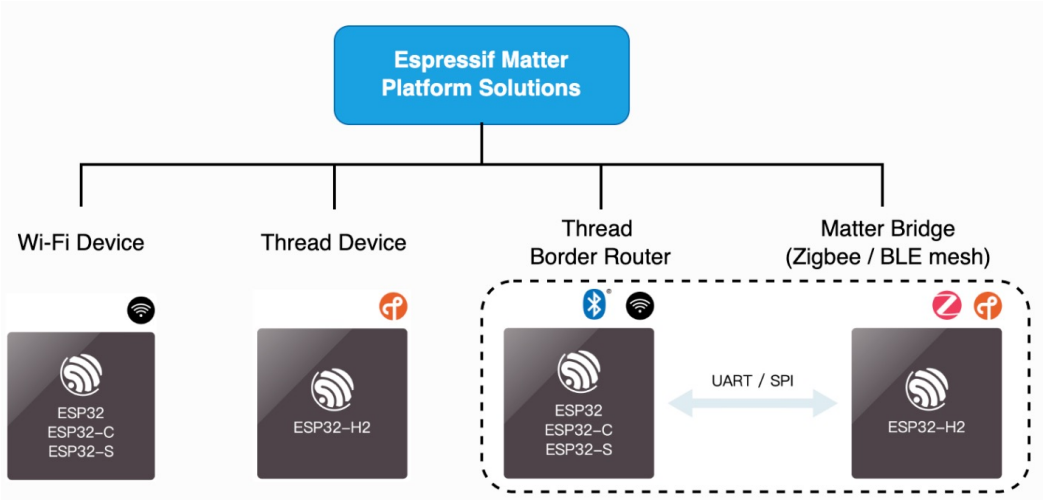


Sample topology



Espressif Matter Solution- Espressif SDK for Matter (ESP32 Arduino Matter is also available)

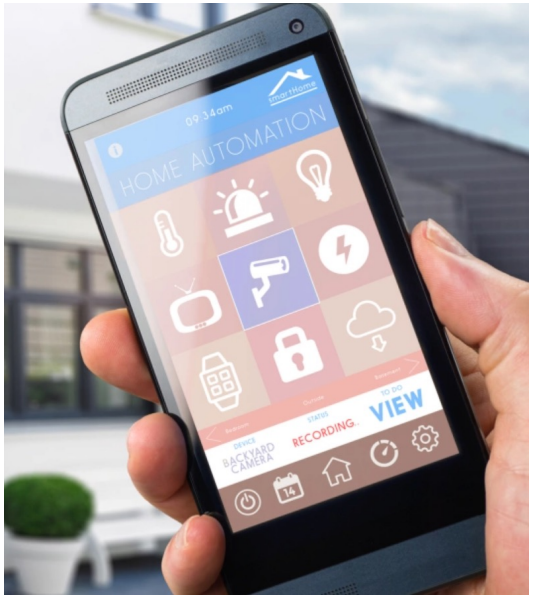


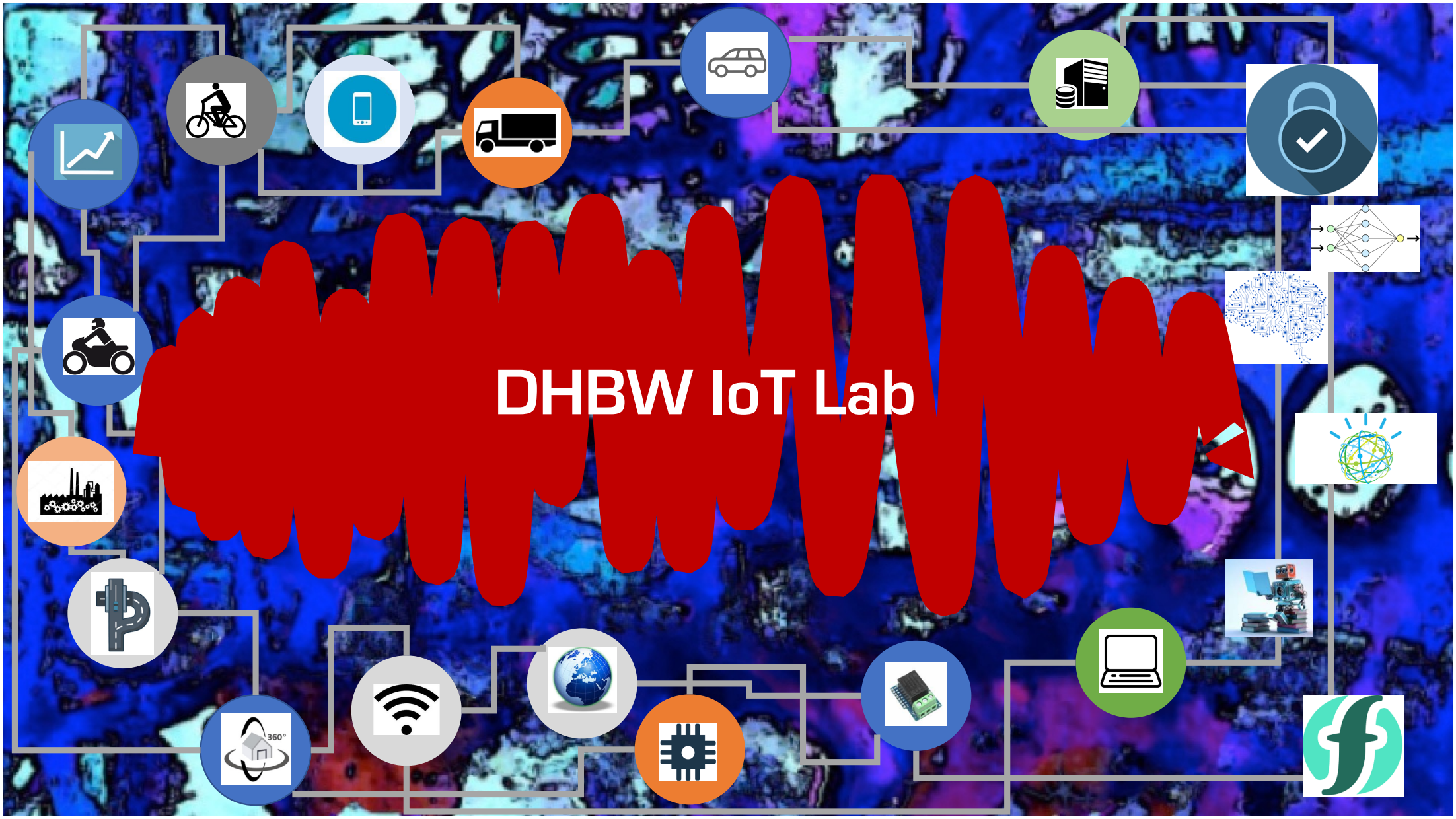


ESP32-H SoCs and modules with 802.15.4 can be used to build **Matter Thread devices**.

Something to try

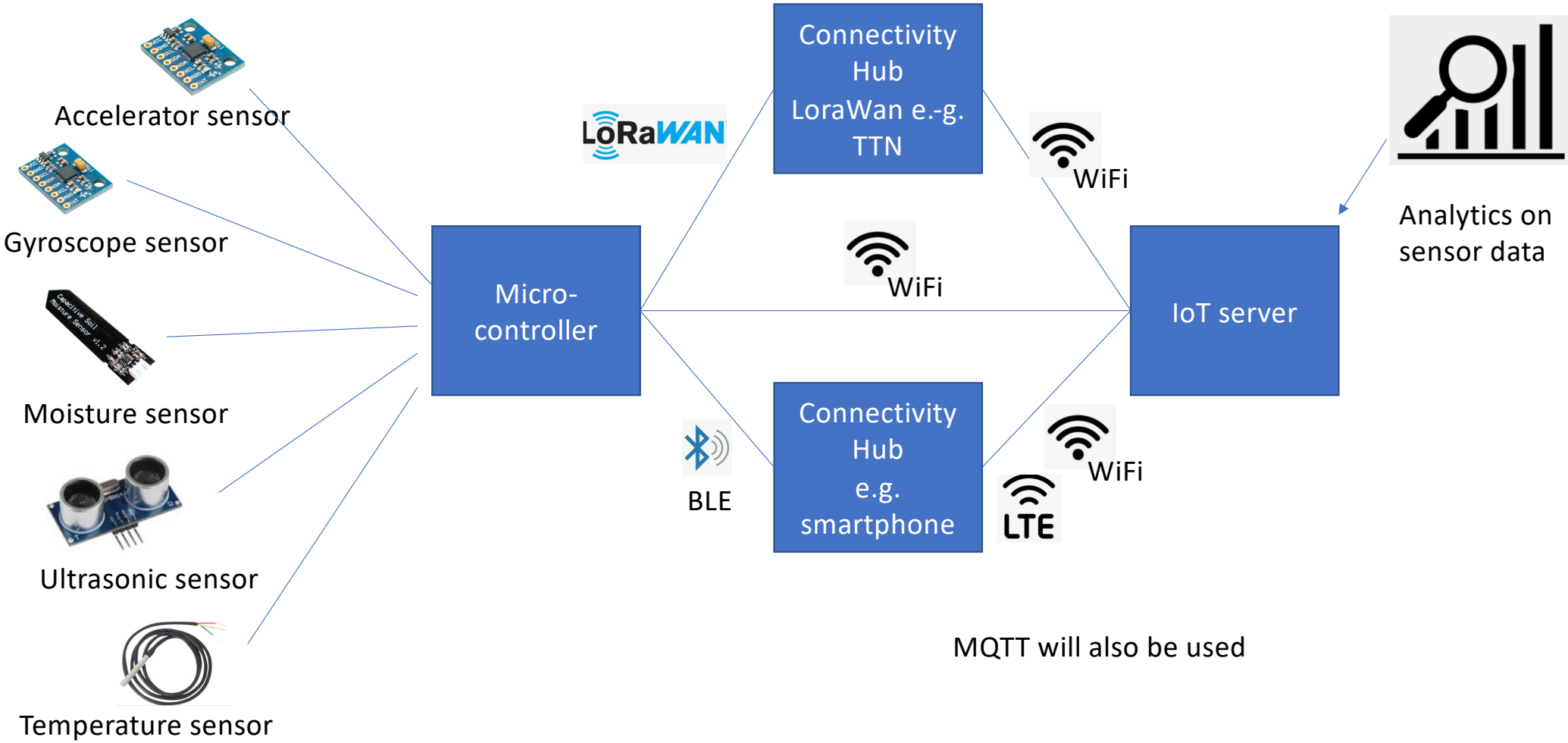
Matter communication




DHBW IoT Lab

Goal of the IoT lab

- **Build a real IoT solution**
  - **Select Hardware and put it together**
  - **Develop a program which reads sensor data, do some pre-calculation, transmit data to a IoT gateway, forward it to Server app.**

- **Get familiar with IoT sensors (from a HW point of view)**
- **Practice some microcontroller programming (which is slightly different from other IT programming)**
- **Get familiar with IoT connectivity topics**
- **Learn how to use different communication protocols (BLE, LoRaWAN, WiFi, MQTT)**
- **Analyse the collected data using analytics functions**

Block diagram IoT Lab